ChatGPT

Blueprint for an AI Skill to Simplify Preferences and Navigate Complex Choices (May 2026)

Bottom-line summary

The proposed AI skill helps users make complex decisions (e.g., choosing a home, designing an app, selecting an academic program) by combining **deterministic decision-analysis frameworks** with **adaptive, flexible elicitation algorithms**. A layered pipeline progressively clarifies values, applies hard constraints, elicits preference weights, prunes dominated alternatives and delivers a ranked shortlist. Each layer uses validated decision science methods: value-focused thinking to surface goals [1], lexicographic "veto" rules to eliminate unacceptable options [2], adaptive preference elicitation (e.g., PAPRIKA, Bayesian experimental design) to learn weights efficiently [3] [4], outranking methods to remove dominated alternatives [5] and simple scoring methods (TOPSIS/WSM) to rank a small shortlist [6]. This scaffold reduces a 1,000-option space to a handful of high-fit choices in ~7–12 interactions, balancing rigor with user comfort.

---

1. Goals and design principles

- **Hyper-personalization through dialogue.** The skill aims to build a **personal decision model** for each user rather than rely on generic rankings. It does this by asking a small number of targeted questions—each chosen to maximally reduce uncertainty about the user's preferences [4].
- **Layered reduction of complexity.** Instead of presenting hundreds of options, the system progressively removes whole branches of the decision tree using different methods at each stage (values elicitation, hard constraints, preference learning, computational elimination, and ranking).
- **Balance deterministic and flexible components.** Deterministic elements (e.g., running an outranking algorithm on a database, computing utility scores from weights) provide reliability and explainability. Flexible components (e.g., natural language dialogue, adaptive question selection) adapt to individual users and domain nuances.
- **Respect cognitive limits.** By asking only a few questions at a time and leveraging pairwise comparisons, the system reduces cognitive load and mitigates choice overload [7].

---

2. High-level architecture and workflow

The skill's logic flow comprises six stages. Each stage outputs a refined set of options and updated user model that feeds into the next stage.

2.1 Stage 1: Values extraction (deterministic + flexible)

1. **Open-ended values questions**: The agent asks 2–3 broad questions to uncover fundamental vs. means objectives. Examples: "What does a great home mean to you?" or "What would make this app feel like a success?" These questions resemble the repertory grid technique's triadic elicitation (asking how two alternatives differ from a third) to surface the user's constructs [8].
2. **LLM-based semantic analysis**: A language model analyses the user's answers, extracting **fundamental objectives** (ends) and **means objectives** (attributes) as suggested by value-focused thinking [1]. For instance, "maintaining close relationships" might map to the attribute "social connectedness."
3. **Deterministic mapping to decision attributes**: The extracted objectives map to a predefined feature schema of the decision domain (e.g., house attributes, app features). This step is deterministic, ensuring that unstructured language is grounded in structured criteria.

Outcome: elimination of irrelevant domains (e.g., ruling out an investment property if the goal is "sense of community").

2.2 Stage 2: Hard constraint elicitation (deterministic)

1. **Veto questions**: The agent asks about non-negotiable constraints (budget ceilings, desired geographic areas, excluded features). These align with fast-and-frugal "take-the-best" or elimination-by-aspects heuristics, which quickly remove options that fail key criteria [2].
2. **Database filtering**: A deterministic filter removes all alternatives that violate these constraints from the options database. This may reduce 1,000 options to 50–100 before any further questioning.

Outcome: a pruned candidate set and a rule set representing the user's hard constraints.

2.3 Stage 3: Adaptive preference elicitation (flexible + deterministic)

1. **Selection of elicitation method**: Depending on the number of criteria and domain complexity, the system chooses between:

2. **PAPRIKA** for small or moderate numbers of criteria, asking pairwise trade-off questions on two criteria at a time and exploiting transitivity to minimize questions [3].

3. **Bayesian Optimal Experimental Design (BOED)** for higher-dimension spaces, where the next question is chosen to maximize expected information gain and the process stops when expected utility loss falls below a threshold [4].
4. **Tournament Tree Method (TTM)** if many alternatives need to be ranked and a reciprocal comparison matrix is required; TTM achieves complete comparison with only m-1 comparisons [9].
5. **Interactive questioning**: The skill presents pairwise comparisons or simple "which do you prefer?" queries in natural language. The LLM translates the selected query into user-friendly language and records the response.
6. **Preference model updating**: A deterministic backend updates the user's weight vector (for additive models) or posterior distribution (for Bayesian models) after each answer, maintaining a measure of uncertainty.

Outcome: estimated weights over relevant attributes and a posterior distribution capturing uncertainty.

2.4 Stage 4: Computational elimination (deterministic)

1. **Outranking analysis**: Using the elicited weights and candidate set, the system runs an outranking method such as ELECTRE to eliminate dominated or incomparable alternatives. ELECTRE determines when one alternative sufficiently outranks another by comparing concordant and discordant criteria [5].
2. **Consistency checks**: If pairwise comparisons are inconsistent (e.g., via AHP's consistency ratio test [10]), the system either prompts the user for clarification or revises weights.

Outcome: further reduction of options (e.g., down to 5–15) without additional user interaction.

2.5 Stage 5: Final ranking and explanation (deterministic + flexible)

1. **Ranking**: Apply a simple scoring method such as the Weighted Sum Method or TOPSIS to rank the remaining alternatives [11] [6].
2. **Justification**: The system generates explanations of how each option scores on key criteria, referencing the user's expressed values. The explanation leverages the LLM's natural language capabilities but is anchored by deterministic scores.
3. **Optional tie-break**: If two alternatives are close, the skill might ask one final clarifying question (a pairwise comparison) before recommending.

Outcome: a ranked shortlist with rationales, enabling the user to choose confidently.

2.6 Stage 6: Progressive profiling and learning (flexible)

1. **Memory of past decisions**: The system stores anonymized preference models for each user (with consent). Over multiple sessions, the number of questions decreases as prior knowledge is reused.
2. **Long-term adaptation**: The agent monitors preference stability; if values drift or context changes (e.g., new life stage), it re-engages with Stage 1 to update values.

Outcome: A continuously improving personal assistant that requires fewer interactions over time.

---

3. Deterministic vs. flexible components

Component	Deterministic (fixed logic)	Flexible (adaptive/heuristic)
Values extraction	Mapping free-text answers into structured criteria and establishing the hierarchy of objectives is deterministic once a vocabulary and ontology exist.	The initial questions and follow-up clarifications are generated flexibly by the LLM to suit the domain and user language.
Hard constraint filtering	Database queries and rule-based elimination are deterministic.	Selection of which constraints to ask first can be adaptive (e.g., based on predicted importance or information gain).

Component	Deterministic (fixed logic)	Flexible (adaptive/heuristic)
Preference elicitation	Updating weight vectors, computing consistency ratios, and selecting an algorithm (PAPRIKA, BOED, TTM) follow deterministic rules.	The specific questions posed are chosen adaptively based on prior responses to maximize information gain [4] ; the language used is generated by the LLM.
Computational elimination and ranking	ELECTRE, TOPSIS, WSM, and AHP weight calculations are deterministic.	Re-engagement for inconsistent answers or optional tie-break questions depends on user responses.
Explanations and conversation	The structure of explanations (mapping scores to user priorities) is fixed.	The narrative phrasing and tone are flexible, delivered by the LLM to match the user's communication style.
Learning over time	Storing and applying past preference models is deterministic.	Deciding when to refresh values or ask new questions is adaptive and may rely on heuristics (e.g., "ask again after X months or if preferences conflict").

4. Algorithmic details and frameworks used

The following algorithms underpin each stage. Their selection depends on the decision domain's complexity and the desired balance between accuracy and user effort.

1. **Natural language understanding & values extraction**: An LLM fine-tuned on values-clarification tasks maps free-text responses to a set of fundamental objectives and means objectives. It uses semantic role labeling and concept extraction to populate the decision model.
2. **Lexicographic filtering**: Fast-and-frugal heuristics apply threshold rules; elimination-by-aspects sequentially removes options failing to meet mandatory criteria [2] .
3. **Preference elicitation algorithms:**
4. **PAPRIKA:** Elicit pairwise trade-off preferences on pairs of criteria; linear programming derives weights and the method uses transitivity to skip redundant comparisons [3] . Suitable when criteria count is moderate and additive models are appropriate.
5. **Bayesian optimal experimental design (BOED):** Maintains a posterior distribution over the user's weight vector and selects the next question to maximize expected information gain. It stops when expected regret or utility loss drops below a threshold [4] . Effective when attribute interactions are complex or uncertainty is high.
6. **Tournament Tree Method (TTM):** In cases requiring a complete reciprocal comparison matrix, TTM generates a consistent matrix using only m-1 comparisons, dramatically reducing cognitive load [9] .
7. **Outranking analysis (ELECTRE)**: Compute concordance and discordance indices for each pair of alternatives using weights; eliminate dominated and incomparable options [5] .

8. **Final ranking**: Use the Weighted Sum Method (WSM) or TOPSIS. WSM normalizes criteria and multiplies by weights to sum scores [11]; TOPSIS computes each alternative's distance to an ideal and anti-ideal point [6]. Both yield clear, interpretable ranks.
9. **Consistency and robustness checks**: For AHP-like comparisons, compute the consistency ratio (CR); if CR > 0.1, prompt the user to revisit comparisons [10]. For regret sensitivity, optionally run minimax regret calculations to ensure recommendations remain robust under preference uncertainty.

---

5. Implementation considerations and user experience

Modularity and integration. Each stage can be implemented as a microservice: an NLU component for values extraction, a constraint engine, a preference elicitation module, an outranking engine, and a ranking/explanation generator. This modularity allows updating algorithms independently.

Data privacy and ethics. The skill should store user preferences securely and allow users to opt out. Since decisions may have ethical or emotional dimensions (e.g., family planning), the system must avoid prescriptive recommendations and present users with transparent rationales.

Engagement and cognitive support. To reduce form abandonment and cognitive load, the interface should display one or two questions at a time, use plain language, and provide progress indicators. Progressive profiling avoids asking the same information repeatedly [7].

Explainability and trust. At each stage, the skill should offer just-in-time explanations of why questions are asked ("This helps us understand what matters most to you") and how recommendations were derived (show weights and scores). Outranking and scoring results should be visualized through simple charts.

Adaptation across domains. While the pipeline is domain-agnostic, domain-specific ontologies and databases are needed. For example, a housing database requires metadata (price, location, size), whereas an academic program database needs fields like discipline, teaching philosophy and cost.

---

6. Conclusion

By blending deterministic decision-science frameworks with adaptive elicitation and natural language processing, the proposed AI skill transforms broad, ill-defined questions into precise, personalized recommendations. It leverages value-focused thinking to articulate what the user truly cares about [1], lexicographic heuristics to quickly discard unacceptable options [2], advanced elicitation techniques like PAPRIKA and BOED to infer utility weights efficiently [3] [4], and outranking and scoring methods to compute and explain final rankings [5]. This layered, modular architecture provides a clear path from a 1,000-option space to a handful of tailored choices, making radical preference simplification practically feasible.

---

[1] Value-Focused Thinking: The Foundation for Decision Quality
https://jpt.spe.org/value-focused-thinking-foundation-decision-quality

[2] Elimination-by-aspects - BehavioralEconomics.com | The BE Hub
https://www.behavioraleconomics.com/resources/mini-encyclopedia-of-be/elimination-by-aspects/

[3] Multi-Criteria Decision Analysis (MCDA/MCDM) | 1000minds
https://www.1000minds.com/decision-making/what-is-mcdm-mcda

[4] ai.stanford.edu
http://ai.stanford.edu/~koller/Papers/Chajewska%2Bal:AAAI00.pdf

[5] [6] [11] A Comparison of Multi-Criteria Decision Analysis Methods for Sustainability Assessment of District Heating Systems | MDPI
https://www.mdpi.com/1996-1073/15/7/2411

[7] Progressive Profiling: Building Comprehensive Customer Identities
https://www.avatier.com/blog/progressive-profiling-building/

[8] Repertory grid technique - EduTech Wiki
https://edutechwiki.unige.ch/en/repertory_grid_technique

[9] The Tournament Tree Method for preference elicitation in Multi-criteria decision-making
https://arxiv.org/html/2510.08197v1

[10] Microsoft Word - AHP-Chapter1.doc
https://superdecisions.com/sd_resources/AHP_Chapter1.pdf